

```

1  /**
2  ****
3  * @file crypt_tools.cpp
4  * @author Robert Myers Jr.
5  * @version V1.0
6  * @brief Helper methods to encrypt and decrypt. See header for more information
7  ****
8  */
9  #include "crypt_tools.hpp"
10
11 namespace {
12     uint32_t round_function(uint32_t R, uint64_t subkey) {
13         // Following similar algorithm here https://www.geeksforgeeks.org/python-feistel-cipher/
14         uint32_t key_part = (uint32_t)(subkey & 0xFFFFFFFF);
15         uint32_t result = R ^ key_part;
16         // cyclic rotate by 3
17         return (result << 3) | (result >> 29);
18     }
19 }
20 uint64_t encrypt(uint64_t plaintext, const std::vector<uint64_t>& subkeys) {
21     uint32_t L = (uint32_t)(plaintext >> 32);
22     uint32_t R = (uint32_t)(plaintext & 0xFFFFFFFF);
23
24     for (int i = 0; i < subkeys.size(); i++) {
25         uint32_t temp = R;
26         R = L ^ round_function(R, subkeys[i]);
27         L = temp;
28     }
29
30     return ((uint64_t)R << 32) | L;
31 }
32
33 uint64_t decrypt(uint64_t ciphertext, const std::vector<uint64_t>& subkeys) {
34     uint32_t L = (uint32_t)(ciphertext >> 32);
35     uint32_t R = (uint32_t)(ciphertext & 0xFFFFFFFF);
36
37     for (int i = (int)subkeys.size() - 1; i >= 0; i--) {
38         uint32_t temp = R;
39         R = L ^ round_function(R, subkeys[i]);
40         L = temp;
41     }
42
43     return ((uint64_t)R << 32) | L;
44 }

```

```

1  /**
2  ****
3  * @file des.cpp
4  * @author Robert Myers Jr.
5  * @version V1.0
6  * @brief Helper methods to preform the light weight DES function
7  * See header for more information
8  ****
9  */
10 #include "des.hpp"
11 #include <stdint>
12 #include <cstdio>
13 #include <cstring>
14 #include <iostream>
15 #include <utility>
16 #include <vector>
17
18 constexpr int round_table = 48;
19 constexpr int pc_1_table_length = 56;
20 constexpr int pc_2_table_length = 48;
21 constexpr int c_and_d_length = 28;
22
23 // Tables were taken from lecture slides
24 constexpr int pc_1_table[pc_1_table_length] = {
25     57, 49, 41, 33, 25, 17, 9,
26     1, 58, 50, 42, 34, 26, 18,
27     10, 2, 59, 51, 43, 35, 27,
28     19, 11, 3, 60, 52, 44, 36,
29     63, 55, 47, 39, 31, 23, 15,
30     7, 62, 54, 46, 38, 30, 22,
31     14, 6, 61, 53, 45, 37, 29,
32     21, 13, 5, 28, 20, 12, 4
33 };
34
35 constexpr int pc_2_table[pc_2_table_length] = {
36     14, 17, 11, 24, 1, 5,
37     3, 28, 15, 6, 21, 10,
38     23, 19, 12, 4, 26, 8,
39     16, 7, 27, 20, 13, 2,
40     41, 52, 31, 37, 47, 55,
41     30, 40, 51, 45, 33, 48,
42     44, 49, 39, 56, 34, 53,
43     46, 42, 50, 36, 29, 32
44 };
45
46 std::pair<uint32_t, uint32_t> apply_pc_1(uint64_t key) {
47     uint64_t c_and_d = 0x0;
48     uint64_t msb_64 = 0x1ULL << 63;
49     uint64_t msb_56 = 0x1ULL << (pc_1_table_length-1);
50
51     int bit_index = 0;
52     int bit_offset = 0;
53     for(int i = 0; i < pc_1_table_length; i++) {
54         bool bit = (key & (msb_64 >> (pc_1_table[bit_index]-1ULL))) > 0;
55         c_and_d |= (msb_56 >> (bit_index))*bit;
56         bit_index++;
57     }
58     auto c = c_and_d >> (pc_1_table_length/2);
59     auto d = c_and_d & 0xFFFFFFFF;
60     // std::printf("C %lb\n", c);
61     // std::printf("D %lb\n", d);
62     return {c, d};
63 }
64
65 std::pair<uint32_t, uint32_t> do_round(std::pair<uint32_t, uint32_t> keys, int s
hift) {
66     if(shift == 1 ||
67        shift == 2 ||
68        shift == 9 ||
69        shift == 16) {
70         shift = 1;

```

```
71     } else {
72         shift = 2;
73     }
74     return {
75         ((keys.first << shift) | (keys.first >> (c_and_d_length - shift))) & 0xF
FFFFF,
76         ((keys.second << shift) | (keys.second >> (c_and_d_length - shift))) & 0
FFFFFFF
77     };
78 }
79
80
81 uint64_t get_subkey(std::pair<uint32_t, uint32_t> keys) {
82     uint64_t subkey = 0x0;
83     uint64_t key = (((uint64_t)keys.first) << c_and_d_length) | keys.second;
84     uint64_t msb_56 = 0x1ULL << (pc_1_table_length-1);
85     uint64_t msb_48 = 0x1ULL << 47;
86     int bit_index = 0;
87     int bit_offset = 0;
88     for(int i = 0; i < pc_2_table_length; i++) {
89         bool bit = (key & (msb_56 >> (pc_2_table[bit_index]-1ULL))) > 0;
90         //std::printf("%d", bit);
91         subkey |= (msb_48 >> (bit_index)) * bit;
92         bit_index++;
93     }
94     return subkey;
95 }
```

```

1  /**
2  ****
3  * @file des_decrypt.cpp
4  * @author Robert Myers Jr.
5  * @version V1.0
6  * @brief Driver program to decrypt messages
7  ****
8  */
9  #include <cstdlib>
10 #include <cstdio>
11 #include <cstring>
12 #include <exception>
13 #include <string>
14 #include <vector>
15 #include "des.hpp"
16 #include "crypt_tools.hpp"
17 int main(int argc, char* argv[]) {
18     if(argc < 3) {
19         std::printf("ERROR: Need 2 arguments(message, key)\n");
20         return 1;
21     } else if(argc != 3) {
22         std::printf("ERROR: Provided too many arguments\n");
23         return 1;
24     }
25
26     std::string hex_str = argv[1];
27     std::string key_str = argv[2];
28
29     uint64_t cipher;
30     uint64_t key;
31     try {
32         cipher = std::stoull(hex_str, nullptr, 16);
33         key = std::stoull(key_str, nullptr, 16);
34     }
35     catch (const std::exception& e) {
36         std::printf("ERROR: Could parse not one of the given argument\n");
37         return 1;
38     }
39
40     auto pair = apply_pc_1(key);
41     std::vector<uint64_t> subkeys;
42
43     auto round_keys = pair;
44     for(int i = 1; i <= 1000; i++) {
45         round_keys = do_round(round_keys, i);
46         auto key = get_subkey(round_keys);
47         subkeys.push_back(key);
48     }
49
50     uint64_t plain = decrypt(cipher, subkeys);
51     std::string message_decrypted;
52     message_decrypted.resize(sizeof(message_decrypted));
53     std::memcpy(message_decrypted.data(), &plain, sizeof(plain));
54     std::printf("Decrypted Hex: %lx\n", plain);
55     std::printf("Decrypted String: %s\n", message_decrypted.data());
56
57 }

```

```

1  /**
2  ****
3  * @file des_encrypt.cpp
4  * @author Robert Myers Jr.
5  * @version V1.0
6  * @brief Driver program to encrypt messages
7  ****
8  */
9  #include "crypt_tools.hpp"
10 #include "des.hpp"
11 #include <cstdint>
12 #include <cstdio>
13 #include <string>
14 #include <cstring>
15 #include <vector>
16 int main(int argc, char* argv[]) {
17
18     if(argc < 3) {
19         std::printf("ERROR: Need 2 arguments(message, key)\n");
20         return 1;
21     } else if(argc != 3) {
22         std::printf("ERROR: Provided too many arguments\n");
23         return 1;
24     }
25     std::string message = argv[1];
26     std::string key_str = argv[2];
27
28     uint64_t key;
29     try {
30         key = std::stoull(key_str, nullptr, 16);
31     }
32     catch (const std::exception& e) {
33         std::printf("ERROR: Could not parse one of the given argument\n");
34         return 1;
35     }
36
37     if(message.length() > 8) {
38         std::printf("ERROR: message is not within 64 bits\n");
39         return 1;
40     }
41
42     auto pair = apply_pc_1(key);
43     std::vector<uint64_t> subkeys;
44
45     auto round_keys = pair;
46     for(int i = 1; i <= 1000; i++) {
47         round_keys = do_round(round_keys, i);
48         auto key = get_subkey(round_keys);
49         subkeys.push_back(key);
50     }
51
52     uint64_t message_data;
53     std::memcpy(&message_data, message.data(), sizeof(message_data));
54
55     uint64_t cipher = encrypt(message_data, subkeys);
56
57     std::printf("Original String: %s\n", message.data());
58     std::printf("Original Hex: %lx\n", message_data);
59     std::printf("Cipher: %lx\n", cipher);
60 }
61

```

```
1  /**
2  ****
3  * @file crypt_tools.hpp
4  * @author Robert Myers Jr.
5  * @version V1.0
6  * @brief Helper methods to encrypt and decrypt.
7  ****
8  */
9  #pragma once
10 #include <cstdint>
11 #include <vector>
12
13 /**
14  * @brief Encrypt a 64 bit message with a list of subkeys
15  */
16 uint64_t encrypt(uint64_t plaintext, const std::vector<uint64_t>& subkeys);
17
18 /**
19  * @brief Decrypt a 64 bit message with a list of subkeys
20  */
21 uint64_t decrypt(uint64_t ciphertext, const std::vector<uint64_t>& subkeys);
```

```
1  /**
2  ****
3  * @file des.hpp
4  * @author Robert Myers Jr.
5  * @version V1.0
6  * @brief Helper methods to preform the light weight DES function
7  ****
8  */
9  #pragma once
10 #include <cstdint>
11 #include <utility>
12 /**
13  * @brief Applys the PC_1 table to the key
14  *
15  * @param key The key used to get the initial C0 and D0
16  */
17 std::pair<uint32_t, uint32_t> apply_pc_1(uint64_t key);
18
19 /**
20  * @brief Does a round to shift Cn and C0
21  *
22  * @param keys The keys that will be shifted
23  * @param shift the number of shifts
24  */
25 std::pair<uint32_t, uint32_t> do_round(std::pair<uint32_t, uint32_t> keys, int s
hift);
26
27 /**
28  * @brief Generates the subkey with a given Cn and Dn key
29  */
30 uint64_t get_subkey(std::pair<uint32_t, uint32_t> keys);
```

```
1 #####
2 # @file CMakeLists.txt
3 # @author Robert Myers Jr.
4 # @version V1.0
5 # @brief CMakeList file to compile Lab 4
6 #####
7 cmake_minimum_required(VERSION 3.15)
8
9 project(Lab4 LANGUAGES CXX)
10
11 # Use C++ 20 features
12 set(CMAKE_CXX_STANDARD 20)
13 set(CMAKE_CXX_STANDARD_REQUIRED True)
14
15 # Generates a compile commands file for LSP
16 set(CMAKE_EXPORT_COMPILE_COMMANDS True)
17
18 add_executable(RFM_encrypt
19     des.cpp
20     des_encrypt.cpp
21     crypt_tools.cpp
22 )
23
24 add_executable(RFM_decrypt
25     des.cpp
26     des_decrypt.cpp
27     crypt_tools.cpp
28 )
```